Introduction to computer programming (2023/2024)

Paolo Franciosa (credits to Massimo Lauria)

Exam April 8, 2024

Working methods: The exam text, the test files of the exercises and a synoptic table on Python syntax can be found in the `exam` folder on your desk, in the "Statistica-Laboratorio" virtual machine. The solutions to the exercises must be placed in the same folder `exam`. **First compile the file** `studente.txt` with the first name, surname and matriculation information

```
nome: Giacomo
cognome: Leopardi
matricola: 123456789
```

Not completing the `student.txt` file is like not handing in the assignment. For the exam, you can only have

- your ID or passport;
- beverages and snacks;

and you must leave your bags and backpacks in the corner of the room, with mobile phones and electronic devices switched off inside.

The grade is the sum of the points of the exercises, so it is possible to get 30 even without completing them all.

Use of test files: tests check very briefly the setting of the solutions and passing the tests does not at all imply that the solution is correct. Read the text of the exercises before starting to solve them. To complete each exercise exercise you must

- write a solution file **with the specified name**;
- have the function **with the specified name** inside;
- place the solution file in the `exam` folder;

- run the `python3 <testfile>` command in that folder

where `<testfile>` should of course be replaced with the name of the appropriate test file for the exercise you are working on.

There is an independent test file for each exercise, so you can work on the exercises one at a time with more ease. The result of each test is a screen (or several screens) showing

- whether the file with the correct name was found,
- whether the file contains the function with the correct name,
- whether calls to the function with different parameter values end by returning correct results.

For each function written, calls are made with different values of the parameters. The result of the tests is returned with the character

- `E` if the call cannot be executed,
- `F` if the function does not return the correct result,
- `.` if the function returns the correct result.

1 Largest value (1 points)

Write a function `largest_value(v_1, v_2)` that:

- takes two numbers `v_1` `v_2` as parameters;
- returns the maximum among the absolute values of `v_1` and `v_2`.

```
print(largest_value(-7, 5))
print(largest_value(4, 8))
print(largest_value(-3, -5.7))
```

```
7
8
5.7
```

Python function must be saved in file: `largest_value.py`

Test file: `test_largest_value.py`

2 Factorial (1 points)

Write a function `factorial(x)` that:

- takes as argument a non-negative integer `x`;
- returns the factorial of `x`, i.e., the product of the first `x` natural numbers.

Examples: - `factorial(3)` returns `6` - `factorial(5)` returns `120`.

The function must be saved in a file named `factorial.py`

Python function must be saved in file: `factorial.py`

Test file: `test_factorial.py`

3 List of larger elements (1 points)

Write a function `list_larger(a, v)` that:

- takes a lists `a` and a value `v`;
- returns a new list containing all item in `a` that are strictly larger `v`.

You can **assume** that `a` contains elements that are comparable with `v`.

Elements in the result must appear in the same order as in `a`.

```
x = [7, 5, 3, 8, -1]
k = 3
print(list_larger(x, k))
```

```
[7, 5, 8]
```

Python function must be saved in file: `list_larger.py`

Test file: `test_list_larger.py`

4 Count equal pairs in a dictionary (1 points)

Write a function `dict_count_equal_pairs(d)` that:

- takes as arguments a dictionary `d`;
- returns the number of items in the dictionary in which the value associate dto the key is equal to the key itself.

For example:

```
d = {"Mario": "Mario", "Gianna" : 8000, "Alberto": 4000, "John": "John"}

print(dict_count_equal_pairs(d))
```

```
2
```

Python function must be saved in file: `dict_count_equal_pairs.py`

Test file: `test_dict_count_equal_pairs.py`

5 Check whether all rows in a matrix are dense (1 points)

Write a function `mat_dense_rows(m, k)` that:

- takes as argument a matrix `m` and an integer `k`;
- returns `True` if each row in `m` contains at least `k` elements different from 0, returns `False` otherwise.

You may assume that `m` is a matrix represented by a list of lists of elements, and `m` contains at least one row and one column.

For example:

```
m = [[1, 2, 5, 0], [0, 3, 0, 5], [3, 0, 0, 0]]
print(mat_dense_rows(m, 2))
print(mat_dense_rows(m, 1))
```

```
False
True
```

Python function must be saved in file: `mat_dense_rows.py`

Test file: `test_mat_dense_rows.py`